Industrial University of Ho Chi Minh City

CQRS Pattern

Nguyễn Trọng Tiến

CQRS là gì?

CQRS (Command Query Responsibility Segregation) là một mẫu kiến trúc phần mềm tách biệt rõ ràng giữa:

Solution

- Command (Ghi) → thay đổi trạng thái hệ thống
- Query (Đọc) → chỉ lấy dữ liệu, không thay đổi gì

Nguyên lý cốt lõi:

“Không nên dùng cùng một model cho cả đọc và ghi”

Example: Hệ thống bán vé tàu

- **Command:**
  - Đặt vé
  - Hủy vé
  - Thanh toán
- **Query:**
  - Xem danh sách vé
  - Xem lịch sử đặt vé
  - Tìm chuyến tàu

CQRS tách riêng:

- Service xử lý đặt vé ≠ Service hiển thị vé

Kiến trúc tổng thể

1. Command Side (Write)

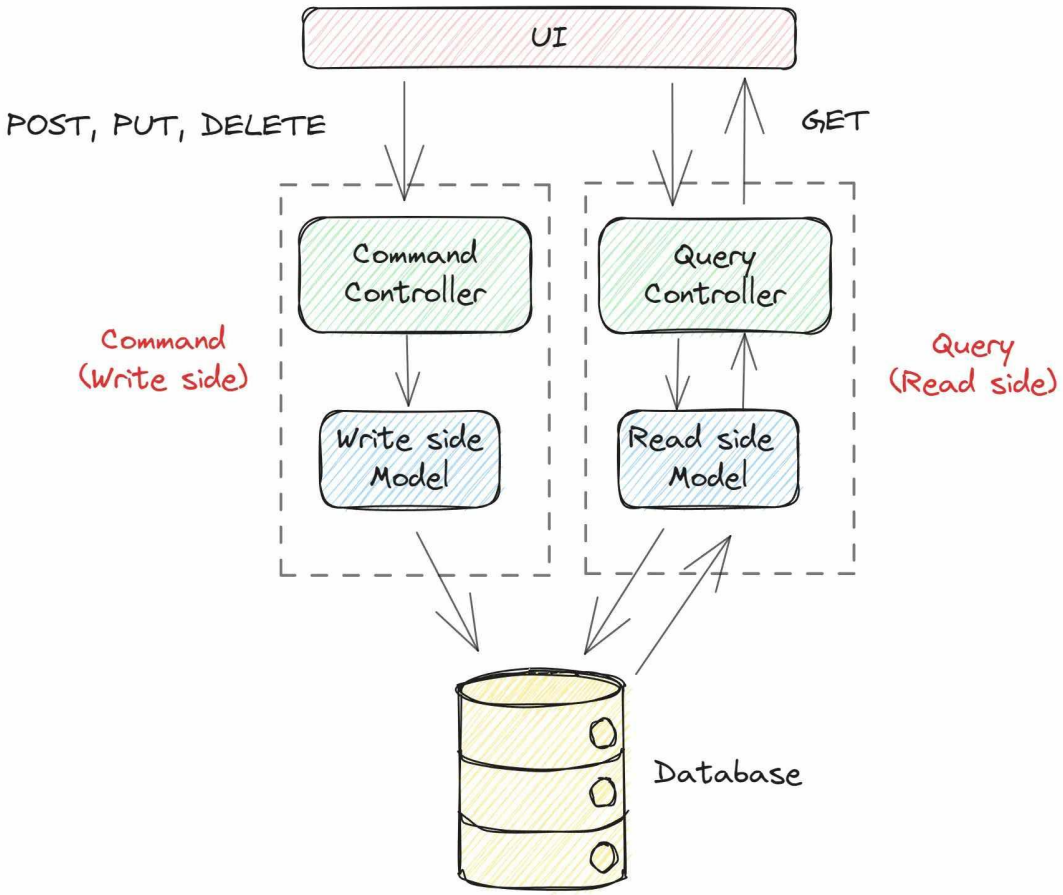
- Nhận request thay đổi dữ liệu
- Validate logic nghiệp vụ
- Ghi vào DB (Write DB)

2. Query Side (Read)

- Chỉ đọc dữ liệu
- Dùng Read DB (được tối ưu riêng)

3. Event / Sync (optional)

- Đồng bộ dữ liệu giữa Write → Read
- Thường dùng:
  - Event Bus (Kafka, RabbitMQ)
  - Event Sourcing



So sánh CRUD vs CQRS

Tiêu chí	CRUD	CQRS
Model	1 model	2 model
Performance	Trung bình	Tối ưu cao
Scale	Khó scale	Scale tốt
Complexity	Thấp	Cao
Phù hợp	App nhỏ	System lớn

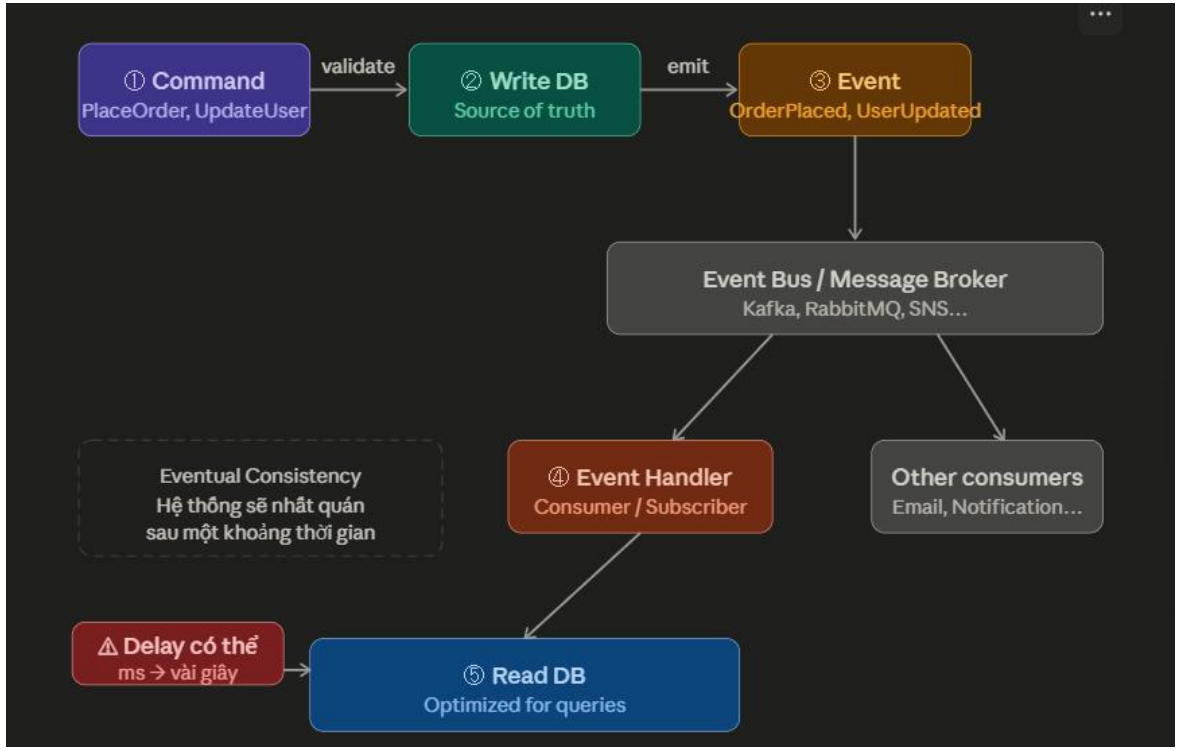
Event-driven + CQRS

Event là gì?

- Flow: Command → Write DB
- Emit Event
- Update Read DB

Giải thích:

- Eventual consistency là gì
- Tại sao Read DB có thể bị delay



Scaling trong CQRS

Scale Read:

- nhiều replica

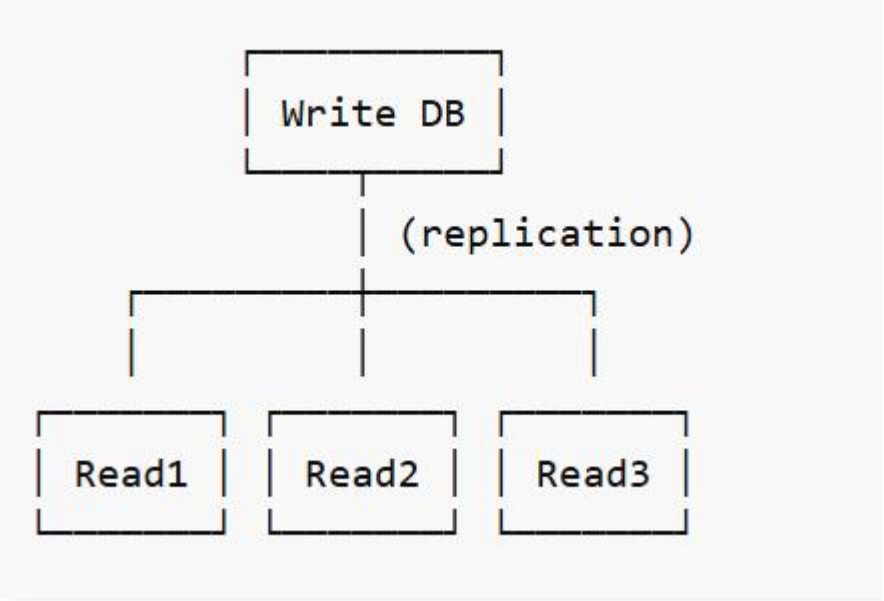
Scale Write:

- shard

Case:

Facebook:

- write ít
- read cực nhiều



CQRS + Microservices

CQRS thường đi với:

- Event-driven architecture
- Microservices

Pattern-mỗi service có:

- DB riêng
- read model riêng

PRACTICE & DESIGN

- Đề bài: Hệ thống bán vé tàu
- Yêu cầu: Thiết kế CQRS cho
  - Đặt vé
  - Hủy vé
  - Xem vé
  - Tìm chuyến

Recommendation

- Command:
  - BookTicket
  - CancelTicket
- Query:
  - GetTickets
  - SearchTrips
- Event:
  - TicketBooked
  - TicketCanceled

Question

- Nếu Read DB bị lag thì sao?
- Có cần transaction không?
- Khi nào không dùng CQRS?

Summary

- CQRS = tách Read / Write
- Tăng performance + scale

Bài 1: Todo App với CQRS

Yêu cầu:

- Xây dựng API:
- **Command:**
  - POST /todos → tạo todo
  - PUT /todos/:id → update
  - DELETE /todos/:id
- **Query:**
  - GET /todos → lấy danh sách
  - GET /todos/:id → chi tiết

Bài 2: Order System (có Event)

Yêu cầu:

- **Command:**
  - Tạo đơn hàng
  - Hủy đơn
- **Query:**
  - Xem danh sách đơn hàng

Bài 3: Train Ticket System

Yêu cầu:

- **Command:**
  - Book Ticket
  - Cancel Ticket
- **Query:**
  - Get Tickets
  - Search Trips

Bài 4: CQRS + SCALING + MICROSERVICE

Tách thành 2 service

Yêu cầu:

- Tách thành:
  - order-command-service
  - order-query-service

Giao tiếp:

  - Kafka / RabbitMQ (giả lập bằng eventBus)

Q&A

Thank you very much!

Industrial University of Ho Chi Minh City

Event Sourcing

Nguyễn Trọng Tiến

Event Sourcing Architecture

- What is Event Sourcing?
- Why use it?
- How it works?

- Keywords: Event, Replay, State

Problem of CRUD

- Only stores final state
- No history
- Hard to debug

- Example: UPDATE account SET balance = 1000

Core Idea

- Do not store state
- Store events

- State = replay events

How It Works

- Command → Event → Store → Projection → UI

Architecture Components

- Command Handler
- Event Store
- Event Bus
- Projection

Replay State

- State is rebuilt by replaying events

Snapshot Optimization

- Use snapshot to avoid replaying all events

CRUD vs Event Sourcing

- CRUD: State
- Event Sourcing: Events
- Better audit and debugging

When to Use

- Banking, E-commerce, Audit systems
- Not for simple CRUD apps

Question

- Large scale but slow?
- → Use snapshot + CQRS + async projection

Event Sourcing - Exercises

- 5 practical exercises from basic to advanced

Exercise 1: Bank Account

- Events: AccountCreated, MoneyDeposited, MoneyWithdrawn
- Store events only (no direct state)
- Implement apply(event), replay(events)
- Output: balance computed from events

Exercise 2: Time Travel / Undo

- Implement undoLastEvent() or getStateAt(index)
- Allow viewing past states
- Hint: replay until index

Exercise 3: Projection (Read Model)

- Separate Write & Read model
- Build accountSummary from events
- Fields: balance, totalDeposits

Exercise 4: Snapshot Optimization

- Create snapshot every N events
- Store snapshot + new events
- Replay from snapshot instead of beginning

Exercise 5: Order System + CQRS

- Events: OrderCreated, ItemAdded, ItemRemoved, OrderConfirmed
- Write side: command -> event
- Read side: projection (totalPrice, status)
- Apply CQRS separation

Question

- What if replay on every query?
- Wrong: replay all events each time
- Correct: use projection + CQRS + async

Industrial University of Ho Chi Minh City

Saga Pattern

Nguyễn Trọng Tiến

Saga Pattern - 3 Hour Lecture

- Distributed transactions in Microservices
- Case Study: Shopee Order Flow

Problem: Distributed Transaction

- Multiple services
- No global ACID
- Need consistency

Why Not 2PC?

- Slow
- Locks resources
- Hard to scale

Saga Pattern Concept

- Split into local transactions
- Each step emits event
- Failure → compensation

Core Components

- Local Transaction
- Event / Message
- Compensation Logic

Saga Flow Example

- Create Order
- Process Payment
- Update Inventory
- Ship Order

Failure Handling

- If step fails → trigger rollback
- Compensation per service

Choreography

- Event-driven
- No central control
- Hard to debug

Orchestration

- Central orchestrator
- Controls flow
- Easier to manage

Compare 2PC vs Saga

- 2PC: Strong consistency
- Saga: Eventual consistency

Shopee Case Study - Overview

- Order Service
- Payment Service
- Inventory Service
- Shipping Service

Shopee Flow

- User places order
- Reserve stock
- Process payment
- Confirm order

Shopee Failure Case

- Payment fails
- Release stock
- Cancel order

Shopee Architecture

- Event Bus (Kafka)
- Microservices
- Async communication

Design Challenges

- Idempotency
- Retry
- Duplicate events

Best Practices

- Use message broker
- Design compensation carefully
- Logging & monitoring

Trap Questions

- Scale but still fail?
- Missing compensation?
- Orchestrator bottleneck?

Summary

- Saga = distributed transaction solution
- Key: events + compensation

Saga Pattern - Practice Exercises

- 6 exercises from basic to advanced
- Focus: Microservices & Distributed Transactions

Exercise 1: Basic Saga (Orchestration)

- Flow: CreateOrder → Payment → Inventory
- Build Saga Orchestrator
- If Payment fails → Cancel Order

Exercise 2: Full Compensation

- Each step has rollback
- Payment → Refund
- Inventory → Restock
- Test failure scenarios

Exercise 3: Choreography

- Event-driven approach
- OrderCreated → PaymentProcessed → InventoryUpdated
- No central orchestrator

Exercise 4: Idempotency

- Handle duplicate events
- Use eventId
- Ignore already processed events

Exercise 5: Retry & DLQ

- Retry failed operations (max 3)
- Push failed events to Dead Letter Queue
- Simulate temporary failures

Exercise 6: Shopee Case Study

- Services: Order, Payment, Inventory, Shipping
- Implement full Saga flow
- Handle failures & compensation
- Add logging & idempotency

Trap Question

- Duplicate payment issue
- Order processed twice
- Find root cause & fix